

APA: Adaptive-Precision Attention

Exact FP16 for the Few Tiles That Matter, FP4 for the Rest

Raphael Friedmann

<https://github.com/kekz1/apa>

October 2026, APA 0.3.0

Abstract

In long-context causal prefill, the softmax mass of a query row concentrates in a small number of key tiles. APA is a causal flash-attention prefill kernel for the RTX 5090 (sm_120a) that exploits this: a first pass runs block-scaled FP4 attention (`mma.sync kind::mxf4nvf4`) over every 64-key tile and flags a tile as *hot* when its share of the running row sum exceeds a threshold ε . Hot tiles skip the FP4 PV product and are recomputed by a second pass in exact FP16; the two partial results are merged by a log-sum-exp. Both passes run in one fused launch with persistent pass-2 workers. A hot tile must not set the reference maximum of the FP4 accumulation: when an attention sink does, the remaining probabilities fall below the FP4 scale range and round to zero. Keeping the cold frame at most 2^{32} below hot maxima raises the worst-case cosine similarity over six Llama-3.2-3B attention dumps from -0.444615 to 0.944326 at unchanged speed. At $\varepsilon = 0.005$ and 122880 keys, 6.0 to 10.3% of active tiles are hot and APA runs $2.50\times$ faster than the same kernel with every tile exact; the speedup grows with context from $1.30\times$ at 8192 keys. Integrated into the imp inference engine (version 0.2.0 of the kernel), dense prefill of a 106451-token prompt drops from 11659.35 ms (FlashAttention-2) to 6433.23 ms ($1.81\times$), with perplexity below FlashAttention-2 with FP16 or FP32 accumulation on Llama-3.2-3B and Qwen3-8B. The kernel is deterministic (bitwise identical across reruns), supports GQA, paged KV caches and chunked prefill with an incremental tile cache, and is released as a header-only CUDA library under MPL-2.0.

1 Introduction

Prefill cost of long prompts is dominated by attention: for a prompt of length N the work is $O(N^2d)$ while the rest of the model is $O(Nd^2)$. FlashAttention [2, 1, 10] removes the memory bottleneck, but the arithmetic remains. Two families of methods attack it. *Low-precision* attention [16, 14, 15] computes $QK^\top$ and PV in INT8, FP8 or FP4 with smoothing and fine-grained scales; it keeps every key but adds error to every score. *Sparse* attention [12, 5, 11] skips keys predicted to carry little mass; it is exact on the

kept keys and wrong on the skipped ones.

APA combines both at tile granularity and lets the data decide the split per query block. Every tile is computed cheaply in FP4, so no key is dropped. The few tiles that carry a large share of the row’s softmax mass, where FP4 error would dominate the output, are routed to an exact FP16 pass. The decision is made online inside the FP4 pass from quantities the pass computes anyway, so it costs one extra small MMA per tile.

Contributions:

- A two-pass attention scheme with an online hot-tile test whose cold verdicts are monotone safe: the running row sum only grows, so a tile judged cold stays below ε of the final row sum (Section 3.2).
- A pass-1 tile order (sink tile first, then the diagonal backwards) that puts high-mass tiles into the running sum early, so the test judges middle tiles against a near-final denominator.
- A cold reference frame decoupled from hot tiles, with a diagnosis of the failure it removes: rows whose output lives in the low-mass tail lost that tail to FP4 underflow next to a dominant sink (Sections 3.2 and 4.1).
- An FP4 pass on sm_120 block-scaled MMA with mean-centred K, per-16-key power-of-two P scales folded into the exponent, a lazy row maximum, and row sums obtained from the same quantized P via an MMA against a ones tile.
- A single fused launch: pass-1 CTAs followed by persistent pass-2 workers that pull work from ticket counters and wait on per-query-block release flags, deadlock-free by ticket order.
- Deterministic statistics and an incremental tile cache for chunked prefill over paged KV.

2 Background

Online softmax. For a query row q and keys k_j , flash attention keeps a running maximum m , row sum $\ell = \sum_j e^{s_j - m}$ and unnormalized output $o = \sum_j e^{s_j - m} v_j$,

Step	Output
stats	per (batch, KV head): K column sums per chunk of 16 tiles, K and V absolute maxima
finalize	K mean $\bar{k}$ (chunks summed in order), head scales g_k, g_v
KV quant	per tile a 9216 B blob: K (mean-centred) and $V^\top$ in E2M1, one UE4M3 scale per 16 values
Q quant	packed Q rows in E2M1 with UE4M3 scales; $q \cdot \bar{k}$ kept for pass 2

Table 1: Prep kernels. Blob = K 4096 B, $V^\top$ 4096 B, K scales 512 B, V scales 512 B, laid out for conflict-free `ldmatrix` (XOR swizzle) and for one bulk copy per tile.

rescaling ℓ and o when m grows [7, 8]. Two partial states (m_1, ℓ_1, o_1) and (m_2, ℓ_2, o_2) over disjoint key sets merge exactly:

$$M = \max(m_1, m_2), \quad w_i = e^{m_i - M}, \quad O = \frac{w_1 o_1 + w_2 o_2}{w_1 \ell_1 + w_2 \ell_2}. \quad (1)$$

APA uses this to merge the FP4 result over cold tiles with the FP16 result over hot tiles. All exponentials run in the $\log_2$ domain (`ex2.approx`).

Block-scaled FP4 on sm_120. The RTX 5090 executes block-scaled MMA (`mma.sync, kind::mxf4nvf4, m16n8k64`) with E2M1 operands and one UE4M3 scale per 16 elements along k (NVFP4, a variant of the OCP microscaling formats [9]). E2M1 represents $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$; UE4M3 has a maximum of 448. APA additionally uses a global FP32 scale per head so that $|x|_{\max}$ maps to $6 \cdot 448 = 2688$.

3 Method

APA computes causal attention for head dim $d = 128$ with GQA (H query heads, H_{kv} KV heads, $G = H/H_{kv}$). Keys are split into tiles of $B_{kv} = 64$. Query rows of the G heads sharing a KV head are *packed*: packed row r is position $\lfloor r/G \rfloor$ of query head $h_{kv}G + (r \bmod G)$, so one CTA of 12 warps covers $12 \cdot 16 = 192$ packed rows and loads each KV tile once for all G heads.

3.1 Preparation

A prep stage converts FP16 inputs into the FP4 layout (Table 1).

Mean-centred K. Subtracting the per-channel key mean, $s_j = q \cdot (k_j - \bar{k})$, shifts every score of a row by the same constant $q \cdot \bar{k}$ and leaves the softmax unchanged [16]. It removes the shared channel offset that would otherwise consume FP4 range. Pass 2 computes exact scores

1. $S \leftarrow \text{MMA}_{\text{fp4}}(Q, K_t)$; apply causal / tail mask
2. candidate max $c \leftarrow m$, or the row max if a score exceeds $m + \tau$
3. per 16-key group: power-of-two scale 2^k , $P' \leftarrow 2^{sc'+b}$ in frame c , $P' \leq 6$
4. $\ell_t \leftarrow \text{MMA}_{\text{fp4}}(P', \mathbf{1})$ tile row sums, frame c
5. if any row has $\ell_t > \varepsilon(2^{m-c}\lambda + \ell_t)$: hot
 - $f \leftarrow \max(m, c - \delta)$; move $o, \ell_{\text{cold}}, \lambda$ to frame f
 - $\lambda \leftarrow \lambda + 2^{c-f}\ell_t$; set bit t in the hot mask, stop
6. cold: move $o, \ell_{\text{cold}}, \lambda$ to frame c ; $\lambda \leftarrow \lambda + \ell_t$
7. $\ell_{\text{cold}} \leftarrow \ell_{\text{cold}} + \ell_t$; $o \leftarrow o + \text{MMA}_{\text{fp4}}(P', V_t)$

Figure 1: Pass 1, one tile t of one warp (c' : score scale, $\delta = 32$, `APA_HOT_DROP`).

$q \cdot k_j$ from the original FP16 keys and subtracts $q \cdot \bar{k}$, so both passes share one logit frame and Eq. (1) applies.

3.2 Pass 1: FP4 attention with a hot-tile test

Each warp owns 16 packed rows; a CTA of 12 warps streams the tile blobs through a 6-stage shared-memory ring filled by `cp.async.bulk` with `mbarrier` completion (thread 0 is the producer). Per tile and warp:

P quantization. The probabilities of a 16-key group are scaled by $1536/2^k$ with $2^k \geq 256 \cdot \max_{\text{group}} P$, so the quantized values satisfy $P' \leq 6$ and the scale 2^k is an exact UE4M3 power of two. The shift is folded into the bias of the exponent, so `ex2` directly produces P' ; the global factor 1536 is divided out in the epilogue.

Lazy maximum. The running maximum m ($\log_2$ domain) is updated, with a full quad reduction and a rescale of o, ℓ_{cold} and λ , only when some score exceeds $m + \tau$ with $\tau = 4$. The bias absorbs the lag, so $P' \leq 6$ still holds; most tiles skip the rescale.

Row sums from the quantized P. Instead of summing FP32 probabilities, pass 1 multiplies the packed P' fragment by a tile of E2M1 ones. The resulting row sums are those of the values that enter PV , so numerator and denominator of the cold partial carry the same quantization.

The hot test. λ accumulates the sums of *all* tiles seen so far, hot ones included, while ℓ_{cold} and o accumulate only cold ones. A tile is hot for the warp if, for any of its 16 rows, its row sum exceeds $\varepsilon\lambda$. The test is warp-uniform (`_any_sync`), so the hot mask has one bit per (warp, tile).

Proposition. Let λ_t be the running sum after tile t and λ_∞ the final sum, both in the frame of the final maximum.

Every tile t that pass 1 marks cold satisfies $\ell_t \leq \varepsilon\lambda_\infty$, and the cold tiles together carry at most $n_c\varepsilon$ of the row’s mass, where n_c is their number.

Proof. A rescale multiplies ℓ_t , λ and all later sums by the same factor, so shares are frame-invariant. Sums of exponentials are non-negative, hence $\lambda_t \leq \lambda_\infty$ and $\ell_t \leq \varepsilon\lambda_t \leq \varepsilon\lambda_\infty$. Summing over cold tiles gives the second bound. $\square$

The per-tile bound is what the test guarantees; the total bound $n_c\varepsilon$ is loose (at 122880 keys, n_c can approach 1920). A hot verdict can be conservative; it costs pass-2 work, not accuracy. The FP4 error of the cold part is bounded only through the per-tile share, which is the mechanism behind the worst-case rows in Section 5.

Cold frame. The frame m sets the scale of every later P' . The per-group exponent k is clamped at -6 so that 2^k stays a normal UE4M3 value; a group whose maximum lies more than about 2^{-10} below the frame therefore quantizes to values under 0.25 and rounds to E2M1 zero. In APA 0.2.0 the frame was the running maximum over all tiles. The sink tile, visited first and usually hot, then set m , and in rows where the remaining probabilities sit near 2^{-20} of the sink every cold tile vanished (Section 4.1). Hot tiles do not need m : their P' is used only for ℓ_t , computed in the candidate frame c , and pass 2 recomputes them. APA 0.3.0 therefore lets a hot tile lift the frame only to $\max(m, c - \delta)$ with $\delta = 32$; λ absorbs the hot sum as $2^{c-f}\ell_t \leq 2^{32}\ell_t$, well inside FP32 range. A cold tile commits its candidate as before, so with $\delta = 0$ the kernel is bitwise 0.2.0. Frames are row-wise; the proposition above holds unchanged, since all sums still share one frame.

Tile order. The denominator λ is small early in the loop, which makes early tiles look hot. Pass 1 therefore visits tile 0 first (the attention sink [12]), then the remaining tiles from the causal diagonal backwards. The tiles that typically carry most mass enter λ first, and the long middle range is judged against a near-final sum.

Epilogue. A warp without hot tiles normalizes and stores its output directly. A warp with hot tiles exports (m, ℓ_{cold}) and the unnormalized cold partial o in FP32 for the merge.

3.3 Pass 2: exact FP16 over hot tiles

A pass-2 group of 4 warps takes the same packed rows as 4 consecutive pass-1 warps. It streams the union of their hot tiles from the original FP16 K and V (flat or paged), double-buffered with `cp.async`: K_{t+1} loads under PV_t , V_{t+1} under QK_{t+1} . Each warp computes only its own hot tiles with `mma.sync.m16n8k16` (f16 inputs, f16 accumulation, the default of imp’s FlashAttention-2 path) and an FP32 online softmax in the log₂ domain. At the

end each row is merged with its pass-1 partial by Eq. (1) and written to O .

The output accumulator stays in f16 across tiles by default. With few hot tiles this is harmless; with all tiles hot ($\varepsilon < 0$), small late contributions next to a dominant sink round away (Section 4.1). `APA_P2_F320=1` sums each 16-token k-step in f16 at full HMMA rate and adds it to an FP32 accumulator; the extra 32 registers per thread cause spills in the 384-thread fused kernel.

3.4 Fused launch

Both passes run in one launch of $n_1 + n_{\text{SM}}$ CTAs of 384 threads, where n_1 is the number of (batch, KV head, query block) pass-1 items. Each CTA takes a ticket from an atomic counter. Tickets below n_1 run pass 1 (causal: heaviest query blocks first); the rest become persistent pass-2 workers whose three 4-warp groups pull (query block, sub-group) items from a second counter. A pass-2 group spins on its query block’s ready flag, which the pass-1 CTA sets with a release store after a fence; the worker reads it with acquire semantics. Tickets are handed to CTAs in the order they start running, so every pass-1 item a worker waits on is already resident: the scheme cannot deadlock. Pass-2 work fills the SMs left idle by pass 1’s last wave.

3.5 Determinism

All floating-point reductions run in a fixed order. In 0.1.0 the K column sums used shared and global float atomics; 5 reruns of prep and attention on the three dumps differed in 539072, 1329742 and 950802 output elements. In 0.2.0 each 16-tile chunk sums its columns in registers, then by XOR shuffles, then over warps in order, and the finalize kernel sums chunks in order. Maxima use integer `atomicMax`, which is order-independent. The same 5 reruns now give bitwise identical outputs; prep time fell from 0.865 to 0.731 ms at 122880 keys.

3.6 Paged KV and chunked prefill

Pass 2 reads K/V through a reader abstraction: flat `[B][S][H_kv][d]` or a paged FP16 pool through a block table [6], plus a flat tail for the chunk not yet in the cache. For chunked prefill, `prefill_incremental` keeps a per-layer `KvState` with the quantized tile blobs and statistics; a new chunk quantizes only its new keys. Head scales are widened by a headroom factor so later keys fit, and a fingerprint of 48 key values detects a stale cache and triggers a full requantization.

4 Evaluation

Setup. RTX 5090 (sm_120a), CUDA 13. Standalone runs use attention inputs dumped from Llama-3.2-3B [4] (24 query heads, 8 KV heads, $d = 128$): 2048 query

Quantity	0.2.0, $\varepsilon = 0.01$	0.2.0, $\varepsilon < 0$
row (position, head)	190, 20	214, 20
mass of the top tile	0.916	0.913
hot tiles, their mass	2, 0.920	1892, 1.000
$\ \text{hot part of ref}\ $	0.2527	0.1197
$\ \text{cold part of ref}\ $	0.3634	0
$\ \text{ref}\ / \ \text{APA out}\ $	0.1670 / 0.2531	0.1197 / 0.1331
$\ \text{ref}\ / \sum_j p_j \ v_j\ $	0.1997	0.1479
cosine	-0.4446	0.8766

Table 2: Worst sampled row of lc_122880_2. Left: FP4 cold part. Right: every tile exact, f16 output accumulation in pass 2.

rows against 122880 keys (lc_122880_{0,1,2}) or 32768 keys (lc_32768_{0,1,2}). Accuracy is the cosine similarity against an FP32 reference on 512 sampled (row, head) pairs; we report mean and minimum. Times are the minimum over 5 repetitions of the mean of 5 launches (CUDA events). Absolute times drift by up to 10% between runs on this GPU (lc_122880_1 at $\varepsilon = 0.005$: 4.707 and 5.200 ms in two runs), so every comparison below is taken within one run. Hot share is the fraction of active (warp, tile) pairs flagged hot. The kernel is deterministic, so accuracy figures do not vary between runs.

4.1 Diagnosing the worst rows

APA 0.2.0 reached a cosine of -0.444615 on one sampled row of lc_122880_2. A diagnostic mode of the benchmark (APA_DIAG=1) decomposes the reference output of the worst row into the contributions of the tiles pass 1 marked hot and cold (Table 2).

In the left column, the sink tile holds 0.916 of the softmax mass, yet the 8% spread over the other 1890 tiles carries an output component (0.3634) larger than the whole reference (0.1670): the two parts nearly cancel, and $\|\text{ref}\|$ is only 0.20 of the probability-weighted mean value norm. The APA output norm 0.2531 equals the hot part alone (0.2527): the cold contribution was lost entirely, the FP4 underflow of Section 3.2. In the right column every tile is exact, yet the cosine is 0.8766: pass 2 accumulates O in f16, and next to the sink each small tile’s contribution falls under half an f16 ulp of the accumulator. Both failures share the pattern of a dominant, low-value sink and a high-value tail.

4.2 Cold frame

Table 3 compares 0.2.0 ($\delta = 0$) with 0.3.0 ($\delta = 32$) in one run. The worst row of each dump improves, from -0.444615 to 0.944326 on lc_122880_2 and from 0.100775 to 0.987198 on lc_32768_2; mean cosines rise on all six dumps. Hot shares change by at most 0.2 points and attention times stay within run-to-run noise. The paged path (block size 16) gives the same accuracy as the flat one; chunked prefill with the incremental tile cache improves

Dump	0.2.0		0.3.0		
	cos min	ms	cos mean	cos min	ms
lc_122880_0	0.898838	4.666	0.999484	0.969733	4.686
lc_122880_1	0.970893	4.883	0.999791	0.997192	4.707
lc_122880_2	-0.444615	4.998	0.999400	0.944326	4.977
lc_32768_0	0.879942	2.360	0.999634	0.967587	2.280
lc_32768_1	0.984021	1.880	0.999906	0.996628	1.891
lc_32768_2	0.100775	1.894	0.999787	0.987198	1.896

Table 3: Cold frame, $\varepsilon = 0.005$, one run. Hot share 0.3.0: 10.3, 9.6, 6.0, 39.5, 23.6, 17.5%; prep 0.731 to 0.735 ms at 122880 keys, 0.161 to 0.162 ms at 32768.

Dump	ε	hot	cos mean	cos min	ms
lc_122880_1	exact	100 %	0.999634	0.988660	12.292
	0.001	28.3 %	0.999876	0.994971	7.794
	0.003	14.1 %	0.999861	0.997483	5.683
	0.01	5.4 %	0.999620	0.993596	4.507
	0.02	2.9 %	0.999392	0.989299	4.184
lc_122880_2	exact	100 %	0.999596	0.876635	12.049
	0.001	19.0 %	0.999835	0.967307	7.489
	0.003	9.1 %	0.999648	0.946039	5.871
	0.01	3.2 %	0.998788	0.944326	4.844
	0.02	1.7 %	0.997822	0.944326	4.340
lc_32768_2	exact	100 %	0.999902	0.986612	3.287
	0.001	47.4 %	0.999942	0.994521	2.953
	0.003	25.6 %	0.999900	0.989024	2.243
	0.01	9.5 %	0.999409	0.985906	1.607
	0.02	4.9 %	0.998656	0.981491	1.301

Table 4: APA 0.3.0 over ε , one run. “exact”: every tile in pass 2 ($\varepsilon < 0$), f16 output accumulation.

its minimum on lc_122880_2 from -0.431078 to 0.822568; 5 reruns stay bitwise identical.

4.3 Threshold and context length

Table 4 shows the trade-off. From $\varepsilon = 0.001$ to 0.02 the hot share falls by a factor of 9.7 to 11.2 and attention time by 1.7 to 2.3 $\times$, while the mean cosine stays above 0.9978. The minimum is flat on lc_122880_2 from $\varepsilon = 0.003$ on (0.946039, then 0.944326), since its worst rows already have only the sink hot. The all-exact run is not the most accurate configuration: its f16 output accumulation gives the lowest minimum on lc_122880_2 (0.876635). With APA_P2_F320=1 the all-exact run of lc_122880_1 reaches a mean of 1.000000 and a minimum of 0.999997 (12.431 to 17.339 ms), while at $\varepsilon = 0.005$ the same option adds 14.5% (lc_122880_1) and 24.9% (lc_32768_1) to the attention time for a mean cosine of 0.999804 instead of 0.999791. It is therefore off by default.

Table 5 shows why the scheme targets long contexts: the hot share falls from 36.0% at 8192 keys to 9.6% at 122880, and the speedup over the all-exact kernel grows from 1.30 to 2.50 $\times$.

keys	exact ms	APA ms	speedup	hot	cos min
8192	0.764	0.589	1.30×	36.0 %	0.996107
16384	1.557	1.114	1.40×	31.5 %	0.994661
32768	3.237	2.025	1.60×	21.3 %	0.988885
65536	6.771	3.594	1.88×	17.6 %	0.990023
122880	13.023	5.200	2.50×	9.6 %	0.997192

Table 5: Context length: first N keys of lc_122880_1 (query rows at $N - 2048$ to $N - 1$), $\varepsilon = 0.005$, APA 0.3.0, one run.

4.4 End to end

APA is integrated into imp [3]; the config key is `attention.apa_eps`. The following numbers were measured with APA 0.2.0, which differs from 0.3.0 only in the cold frame and runs at the same speed (Table 3); 0.3.0 changes the outputs of rows with a dominant sink, so its perplexity has to be measured again. Llama-3.2-3B Q8_0, 106451-token prompt, `--max-seq-len 108000`, prefill time:

Mode	FA2 ms	APA ms	speedup
default (sparse prefill)	6661.95	5241.18	1.27×
dense	11659.35	6433.23	1.81×

Table 6: imp prefill, APA 0.2.0, $\varepsilon = 0.005$. Dense: `sparse_prefill_topk_tokens=0`. Speedup covers the whole prefill, not only attention.

Perplexity. Table 7, imp on ppl_corpus_45k (13k to 14k tokens) for Llama-3.2-3B and Qwen3-8B [13] in Q8_0, APA 0.2.0 ($\varepsilon = 0.002$ and 0.01 rows: 0.1.0, which 0.2.0 reproduces exactly at $\varepsilon = 0.005$). The two FlashAttention-2 variants differ only in the accumulation type and span 17.3644 to 17.3732 (Llama) and 10.7974 to 10.8110 (Qwen). Every APA setting with $\varepsilon > 0$ lies below both; the all-exact run is 0.0009 above FA2 f32 on Llama. Raising ε from 0.002 to 0.01 cuts default-mode prefill from 5915.15 to 4863.56 ms.

5 Limitations

- **Mass is not output.** The hot test bounds each cold tile’s share of the softmax mass, not of the output. The remaining worst row of lc_122880_2 (cos 0.944326) has the same shape as Table 2: two hot tiles with 0.921 of the mass, a cold part of norm 0.3109 against a reference of 0.1488, so FP4 score error on the tail decides the result. A test weighted by the value norm of a tile, or a bound on the accumulated cold mass, would target such rows.
- **Chunked prefill.** The incremental tile cache widens head scales for later keys; its minimum on

Variant	Llama 3B	Qwen3 8B	prefill ms
FA2, f16 acc.	17.3644	10.7974	6661.95
FA2, f32 acc.	17.3732	10.8110	
APA, all exact	17.3741	10.7868	
APA $\varepsilon = 0.002$	17.3556	10.7855	5915.15
APA $\varepsilon = 0.005$	17.3423	10.7869	5241.18
APA $\varepsilon = 0.01$	17.3385	10.7728	4863.56

Table 7: Perplexity (lower is better) and default-mode prefill time of the 106451-token prompt (Llama), APA 0.2.0 / 0.1.0.

lc_122880_2 is 0.822568 against 0.944326 for a full requantization.

- **Exact mode.** The f16 output accumulation of pass 2 limits the all-exact configuration; the FP32 alternative costs 14.5 to 24.9 % at $\varepsilon = 0.005$ through register spills.
- **Granularity.** The hot verdict is per (warp, tile): one row of 16 makes the tile exact for all.
- **Scope.** Head dim 128, causal, FP16 inputs, sm_120a only.
- **Evidence.** Six dumps from one model; end-to-end numbers are from APA 0.2.0. A direct comparison with FP4 attention kernels such as SageAttention3 [15] on the same inputs is open.

6 Related work

FlashAttention [2, 1, 10] established IO-aware exact attention; FlashAttention-3 adds FP8 on Hopper. SageAttention [16, 14, 15] quantizes Q, K, P, V to INT8, INT4, FP8 and FP4 with K smoothing by mean subtraction, which APA adopts; it applies one precision to every tile. StreamingLLM [12] keeps sink and recent tokens, MInference [5] predicts sparse patterns for prefill, Quest [11] selects pages by query-aware bounds for decode. These skip keys; APA keeps every key and spends precision instead of dropping work, with the split decided online from the FP4 pass itself.

7 Conclusion

Long-context attention is skewed enough that exact arithmetic on 6 to 10 % of the tiles at 122880 keys, FP4 on the rest, and an exact log-sum-exp merge run 2.50× faster than an all-exact kernel. The skew that makes this pay also creates the main hazard: a dominant sink can push the informative tail out of the FP4 range. Keeping the cold frame independent of hot tiles removes that failure at unchanged speed. The code is available at <https://github.com/kekz1/apa> under MPL-2.0.

References

- [1] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] Raphael Friedmann. imp: LLM inference engine. <https://github.com/kekz1/imp>, 2026.
- [4] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [5] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.
- [7] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [8] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- [9] Bitan Darvish Rouhani et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.
- [10] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [11] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.
- [12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [14] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In *International Conference on Machine Learning (ICML)*, 2025.
- [15] Jintao Zhang, Jia Wei, Pengle Zhang, Xiaoming Xu, Haofeng Huang, Haoxu Wang, Kai Jiang, Jun Zhu, and Jianfei Chen. SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-bit training. *arXiv preprint arXiv:2505.11594*, 2025.
- [16] Jintao Zhang, Jia Wei, Pengle Zhang, Jun Zhu, and Jianfei Chen. SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration. In *International Conference on Learning Representations (ICLR)*, 2025.